

Phase 2: Geography, Step by Step

Oct 5, 2026 · @Caroline Ogbara Edoghotu

What Phase 2 delivers

Phase 2 puts Rivers State's real map shapes into the database and builds the reusable import pipeline that every later dataset will go through. When it is done, the platform knows Nigeria, Rivers State, its 23 LGAs and 318 wards, and can answer "which LGA and ward is this point in?".

The Backend Standards (section 17) say Phase 2 is done when:

- All 23 LGAs can be queried with valid shapes.
- A GiST spatial index is in place on the shape column.
- The point-in-polygon test (`ST_Contains`) passes.
- The import path (file, staging, checks, live) works end to end using the `lga_boundaries` sheet in `rivers_state_rainfall_and_boundaries.xlsx`, read straight from Excel with no manual conversion to GeoJSON.

On the frontend, Phase 2 gives you the first real screen: the home map showing the state and its LGAs.

Before you start

Phase 2 depends on Phase 1, so check these work first: the backend runs with `npm run dev`, Firebase sign-in works, the `users` table exists with roles, and the admin-only role check (`authorise("admin")`) refuses everyone else. PostGIS must be switched on in your database.

These are the files you will load, all already in your ClimateSoftware folder:

File	What is inside	Shape type	Used for
<code>rivers_state_rainfall_and_boundaries.xlsx</code> , sheet <code>lga_boundaries</code>	23 LGAs, 14 columns, shapes written as WKT text in the last column	Polygon (as text)	The official Phase 2 test: first run of the import pipeline
<code>rivers_state_boundary.geojson</code>	1 state outline (geoBoundaries, <code>NG-RI</code>)	Polygon	Rivers State row (level 1)

File	What is inside	Shape type	Used for
<code>rivers_state_lgas.geojson</code>	23 LGAs (geoBoundaries), with <code>lga_name_standard</code>	Polygon	Cross-check of the Excel import
<code>rivers_state_wards.geojson</code>	318 wards (GRID3/INEC), with <code>wardname</code> , <code>wardcode</code> , <code>lga_name</code>	Polygon	Ward rows (level 3), second run of the pipeline

In the `lga_boundaries` sheet, two frozen rows sit above the 23 data rows, so check which of them holds the column names and have the importer start from there.

What you provide when execution starts

Have everything below ready before step A. Files go in through the admin pages; keys go in the `.env` files, never in Git.

Data files (uploaded on the import screen)

What	File	Where it goes
State outline	<code>rivers_state_boundary.geojson</code>	Level 1 (State) under Nigeria
23 LGA boundaries	<code>rivers_state_rainfall_and_boundaries.xlsx</code> , sheet <code>lga_boundaries</code>	Level 2 (LGA), the official Phase 2 test
318 ward boundaries	<code>rivers_state_wards.geojson</code>	Level 3 (Ward)
LGA cross-check	<code>rivers_state_lgas.geojson</code>	Not imported; used to compare LGA names and shapes

Details you type into the admin forms

- Country: name Nigeria, code NGA, levels State, LGA, Ward.
- Data sources: provider name, website link, licence and download date for geoBoundaries and for GRID3/INEC. Copy these from `README_rivers_state_boundaries.md` and `README_rivers_state_wards_settlements.md`.

Accounts and keys

Item	Used by	Goes in
Firebase web config (apiKey, authDomain, projectId, appId)	Frontend sign-in	<code>frontend/.env</code>
Firebase service-account JSON file	Backend token checks	<code>backend/.env</code> points to the file path
Mapbox access token	Frontend map	<code>frontend/.env</code> as <code>VITE_MAPBOX_TOKEN</code>
Backend address, for example <code>http://localhost:4000/api/v1</code>	Frontend calls	<code>frontend/.env</code> as <code>VITE_API_BASE_URL</code>
PostgreSQL connection (host, port, database, user, password) with PostGIS on	Backend	<code>backend/.env</code> as <code>DATABASE_URL</code>
Your email for the first admin	Backend	<code>backend/.env</code> as <code>BOOTSTRAP_ADMIN_EMAIL</code>

Reference documents for whoever writes the code

- Backend Engineering Standards and Frontend Engineering Standards (the rules every file follows).
- Approach, Architecture & Backlog2 (the table design).

Your answers to the four decisions at the end of this doc.

Not needed for Phase 2: the CHIRPS rainfall CSVs (Phase 5), the population JSON, the settlement names file, and `climate_platform_sample_data_DEMO.xlsx`, which is demo data and must never be imported as real.

Backend steps

Work through these in order. Each step ends with something you can test before moving on.

A. Tables (database)

1. Create the `countries` entity and migration: `country_code` (ISO code, `NGA`), `country_name`, an optional MultiPolygon shape, `source_id`. No seed rows: the table starts empty and an admin adds each country on the admin Countries page. Also give each country a list of its level names (for Nigeria: State, LGA, Ward), so the import screen knows which levels that country has.

2. Create the `admin_units` entity and migration: `UUID id`, `country_code`, `parent_id` (points to another admin unit), `level` (1 state, 2 LGA, 3 ward), `level_name` ("State", "LGA", "Ward"), the name, `unit_code` (the source's own code, for example `wardcode`), a MultiPolygon shape in SRID 4326, and `source_id`.
3. In a separate migration, add the GiST index: `CREATE INDEX idx_admin_units_geom ON admin_units USING GIST (geom);` plus normal indexes on `parent_id` and `level`.
4. Add the link from `users.scope_admin_unit_id` to `admin_units`, now that the table exists. This is what makes "a responder can only edit their own region" work.

B. Repository (the only code that talks to the database)

5. `AdminUnitRepository` extends `BaseRepository` and adds: `findChildren(parentId)`, `findByLevel(level)`, `findContaining(lon, lat, level?)` using `ST_Contains`, and `findGeometry(id)` which returns one shape only. List queries never return shapes.

C. Import pipeline (built once, reused in Phase 5 and later)

6. Create an `import_runs` table (who uploaded, file name, type, status, row count, error count, date) and an `import_staging_rows` table that holds each raw row plus its error messages.
7. Write file readers: XLSX with `exceljs`, plus GeoJSON. Shapes are accepted as GeoJSON or as WKT text, so read the WKT column in the Excel sheet directly. Also build the template generator with `exceljs`: for a chosen country and level it produces an Excel file with an "Areas" sheet (columns ``unit_name``, ``unit_code``, ``parent_code``, ``geometry_wkt``), an "Instructions" sheet in plain words with one filled example row, and a "Parent areas" sheet listing the valid parents already in the database (for LGAs, the states; for wards, the 23 LGAs), so people copy exact names and codes instead of typing them. When an uploaded file's headings differ from the template, as the existing ``lga_boundaries`` sheet's do, the admin matches its columns to the template's on screen, so real files still import without manual conversion.
8. Write the checks that run on staging rows: shape is valid (`ST_IsValid`, fix with `ST_MakeValid`), shape is inside Nigeria's rough box, name is present, parent can be found, and the name matches the standard list (Emuoha = Emohua, Omumma = Omuma, Obia/Akpor = Obio-Akpor). Turn every Polygon into MultiPolygon so all rows are the same type.
9. Write promotion: one database transaction that copies only the rows that passed into `admin_units`, then marks the run "promoted". If anything fails, nothing is copied.

D. Load the real data, all through the admin pages (no seed scripts, no SQL by hand)

10. On the admin Countries page, create Nigeria (code NGA, levels State, LGA, Ward). Then upload the state outline on the import screen as a level-1 row under Nigeria.

11. Run the `lga_boundaries` sheet on the import screen: 23 LGAs at level 2, each with Rivers State as parent. This is the official Phase 2 test.
 12. Run `rivers_state_wards.geojson` on the same import screen: 318 wards at level 3, each linked to its LGA by `lganame`.
- E. Endpoints** (all under `/api/v1/`, each with a Zod schema, a thin controller and a service with unit tests)

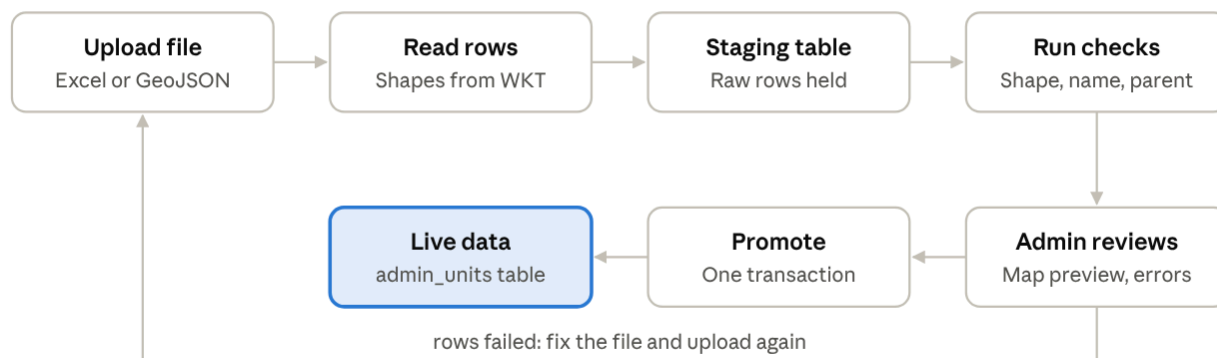
Method and path	Who can call it	What it returns
GET /countries	Everyone	List of countries with their level names
GET /countries/:code	Everyone	One country with how many areas it has at each level
POST /countries	Admin	Creates a country: code, name, level names
PATCH /countries/:code	Admin	Edits the name or level names
GET /admin-units?level=2&parentId=	Everyone	Paged list of names, codes and levels, no shapes
GET /admin-units/:id	Everyone	One unit with its parent and child count
GET /admin-units/:id/children	Everyone	The wards of an LGA, or the LGAs of the state
GET /admin-units/geojson?level=2&bbox=	Everyone	Shapes for the map, only inside the visible area
GET /admin-units/locate?lon=&lat=	Everyone	The state, LGA and ward that contain a point
GET /imports/templates/admin-units?country=NGA&level=2&format=xlsx	Admin	Downloads the blank template for that country and level, as Excel or as a sample GeoJSON
POST /imports	Admin	Upload a file for a chosen country and level; creates a run and fills staging
GET /imports/:id	Admin	Run status plus row errors
POST /imports/:id/promote	Admin	Moves passed rows into the live table

13. Add every request to the Postman collection, including the error cases (bad file, non-admin user, point outside Rivers State).
14. Run `npm run typecheck`, then write `docs/admin-geography.md` and `docs/data-import.md` in plain language.

How the import pipeline works

Nothing reaches the live table until it passes the checks and an admin promotes it

The same pipeline loads LGAs and wards now, and rainfall in Phase 5



import pipeline · 7 steps, 1 loop back

A file never writes straight into the live table. Its rows wait in staging, get checked, and only go live when an admin presses Promote; failed rows send the admin back to fix the file.

Frontend steps and pages

Start the frontend once the backend endpoints above answer correctly in Postman. Follow the order from Frontend Standards section 17.

1. Write the types in `types/admin-unit.types.ts` and `types/import.types.ts`, matching the backend replies field for field.
2. Write `endpoints/admin-unit.endpoints.ts` and `endpoints/import.endpoints.ts`, one function per endpoint, plus a country endpoints file for the country pages.
3. Write the hooks: `useAdminUnits`, `useAdminUnit`, `useAdminUnitChildren`, `useAdminUnitLayer(bbox, level)` (waits 300 ms after the map stops moving, then fetches only the visible area), `useLocatePoint`, `useImportRun`.
4. Build the map pieces in `components/map/`: `ClimateMap`, `LayerToggle` (lists only State, LGA and Ward for now), `MapLegend`, `AdminUnitPopup`, `SourceBadge`.
5. Build the pages below and wire their routes.
6. Write Cypress tests with fake network replies for each page, check file sizes, then write `docs/features/admin-geography.md`.

Pages Phase 2 needs

Page	Route	Who	What it shows
Home map	/	Everyone	Full-screen map centred on Rivers State, state outline and 23 LGAs, LGA names on the map, layer switcher, legend, zoom in to see wards. If no country exists yet, admins see a "Set up your first country" button
LGA directory	/places	Everyone	Searchable list of the 23 LGAs, each with its ward count, linking to its profile
LGA profile	/places/:id	Everyone	The LGA outlined on a small map, its wards listed and clickable, its code and source. Later phases add rainfall and flood history here
Ward view	/places/:id/wards/:wardId	Everyone	One ward on the map with its parent LGA and source
Admin: countries	/admin/countries	Admin	Every country, its levels and how many areas each level has
Admin: add or edit country	/admin/countries/new	Admin	Form for code, name and level names (add, rename or reorder levels)
Admin: country detail	/admin/countries/:code	Admin	Its areas by level, with an "Import areas" button that opens the import screen for that country

Page	Route	Who	What it shows
Admin: import data	<code>/admin/imports/new</code>	Admin	Choose the country, the level and the data source; a "Download template" button gives the right Excel or GeoJSON file for that choice; then upload the filled file and match its columns if the headings differ
Admin: review import	<code>/admin/imports/:id</code>	Admin	Rows that passed and failed with reasons, a preview map with failed shapes in red, and a Promote button
Admin: import history	<code>/admin/imports</code>	Admin	Every past import with who ran it, when, rows and status

Small touches that make Phase 2 feel special

- **"Where am I?" button.** Uses the browser's location (with permission) and calls `/admin-units/locate` to show the ward and LGA you are standing in. It is the simplest proof the point-in-polygon search works.
- **Click anywhere.** Clicking the map shows a popup with the state, LGA and ward for that spot.
- **Shareable map links.** The current zoom, position and selected LGA live in the web address, so a link opens the same view.
- **Preview before promote.** Admins see staged shapes on a map before anything goes live, so a shape in the wrong place is caught by eye.

Decisions to make first, and the done checklist

Four small decisions block step A. My recommendation is given for each.

- **Sources come in Phase 3, but these rows need one now.** `admin_units.source_id` points at `data_sources`, which the Standards only build in Phase 3. Recommended: create a minimal `data_sources` table in Phase 2 with a small "add source" form on the import screen, so geoBoundaries and GRID3 are entered through the UI rather than seeded. Phase 3 then adds the full sources features.

- **Column names differ between documents.** The Standards entity uses `name` and `boundary`; the Backlog2 diagram uses `unit_name` and `geom`, plus `country_code`, `unit_code` and `source_id`. Recommended: follow Backlog2, since it has the extra columns the rules require, and note the choice in `docs/admin-geography.md`.
- **Ward names are placeholders.** The wards file marks its rows "Operational Placeholder" and names Obio/Akpor's wards "Obward 1" to "Obward 17". Recommended: load them as they are, keep `wardcode` as `unit_code`, and show the source on screen. Also note the wards file spells LGAs differently again ("Obio/Akpor", "Port-Harcourt", "Akuku Toru"), so the name-matching list must cover those too.
- **The very first admin is the only thing the UI cannot create.** Everything else (countries, sources, LGAs, wards) is entered by an admin, but someone has to be admin first. Picked: put your email in the backend `.env` as `BOOTSTRAP_ADMIN_EMAIL`; when that email signs in for the first time, its `users` row is created as admin. Every later admin is promoted on the admin Users page. This is safer than "the first person to sign in becomes admin", which anyone could win on a live server.

Phase 2 done checklist

- ☐ `countries` and `admin_units` migrations run on a fresh database, with no seed rows
- ☐ GiST index on the shape column confirmed with `\d admin_units`
- ☐ `lga_boundaries` sheet imported through staging and promoted: 23 LGAs, no manual conversion
- ☐ Nigeria created on the admin Countries page, then the state row and 318 wards loaded through the same pipeline
- ☐ Every row has a `source_id`
- ☐ `locate` returns the right LGA for a point in Port Harcourt and an error for a point outside Nigeria
- ☐ List endpoints return no shapes; the map endpoint needs a `bbox`
- ☐ Non-admins get refused when adding or editing a country, and on all `/imports` routes
- ☐ Service unit tests pass and `npm run typecheck` is clean
- ☐ Postman folder "Admin Geography" covers every success and error, and a downloaded template, filled in and uploaded, imports cleanly
- ☐ Home map, LGA directory, LGA profile, the three admin country pages and the three admin import pages work, with Cypress tests
- ☐ `docs/admin-geography.md` and `docs/data-import.md` written